School of Computer and Systems Sciences
Jawaharlal Nehru University
CS - 403, Data Structures
Time: 1 hr. 30 mins. MCA, 1st Sessional Exam. 2024 Full marks: 10x3 = 30

Attempt any three (including subparts)

1 a) What is the difference between direct and indirect recursion ? What is tail recursion and what is its advantage?

b) Write a recursive (but not tail-recursive) algorithm (using ***C*** pseudocode) to find the factorial of a given number ***n*** (where the value of ***n*** is provided by the user as input) and specify the number stack frames (of the call stack) required for executing the function implementing this algorithm.

c) Write an algorithm (using C pseudocode) to store a list of ***n*** elements taken from the user as input (where the value of ***n*** is taken from the user as input) and perform any of the following operations depending upon user's choice.

i) Insert an element at the ***k***th position in the list (where the value of ***k*** is taken from the user as input)

ii) Sort the list of elements (stored in the array) in ascending order.

(2+3+5)

2 a) Write a tail-recursive algorithm (using ***C*** pseudocode) which takes the value of ***n*** from the user as input and displays the **n**th term of the Fibonacci series as output. Also specify the number stack frames (of the call stack) required (for the compiler which recognizes tail-recursion and implements it) for executing the function implementing this algorithm.

b) A complex-valued matrix ***X*** is represented by a pair of matrices (A, B), where ***A*** and ***B*** contain real values. Write an algorithm (using ***C*** pseudocode) that computes the product of two complex-valued matrices (A,B) and (C,D), where (A, B) * (C, D) = (A+iB) * (C+iD) = (AC-BD) + i(AD+BC). Determine the number of additions and multiplications if the matrices are all n x n.

c) Explain (by specifying suitable examples of programming languages) **0** - based indexing, 1-based indexing and **n**-based indexing for arrays.

(3+4+3)

3 a) Write down algorithms (using ***C*** pseudocode) to perform the following operations on a stack (implemented using array) of size ***n*** (where the value of ***n*** is provided by the user as input).

i) Push an element (provided by the user as input) to the stack only if there is no overflow.

ii) Pop ***k*** number of elements (provided by the user as input) from the stack (where the value of ***k*** is also taken from the user as input) only if it is non-empty.

b) Write an algorithm (using *C* pseudocode) to perform the addition of two polynomials [where each polynomial is stored the array as a list of elements in the form (m, e_{m-1}, b_{m-1}, e_{m-2}, b_{m-2}, … , e_0, b_0) where, the first entry ***m*** represents the number of non-zero terms followed by two entries (e_i, b_i) for each non-zero term (representing an exponent-coefficient pair) in the order $e_{m-1} > e_{m-2} > \ldots > e_0 >= 0$] and print the resultant polynomial.

(4+6)

4 a) Let an array be represented as A(2:20, 3:30, 1:25, 0:25, 5:30). Assuming row major representation of the array (using sequential byte addressing scheme) with 2 bytes per element. If ***k*** is the address of A(4,5,6,7,5) then the address of the element represented as A(5,10,15,6,12) is ____________________ [Fill in the blank correctly].

b) Write down algorithms (using *C* pseudocode) to perform the following operations on a non-circular queue (implemented using array) of size ***n*** (where the value of ***n*** is provided by the user as input).

i) Insert ***k*** number of elements (provided by the user as input) to the non-circular queue (where the value of ***k*** is also taken from the user as input) only if there is no overflow.
ii) Delete an element from the queue only if it is non-empty
iii) Reverse the elements of the non-circular queue using stack

(4+6)

5 a) Let a sparse matrix be stored in an array X(0:t,0:2) where ***t*** is the number of nonzero terms. The elements X(0,0) and X(0,1) contain the number of rows and columns of the matrix respectively. X(0,2) contains the number of nonzero terms. Let X(i,0), X(i,1) and X(i,2) contain the row number of the element, the column number of the element and the value of the non-zero element in the matrix respectively, for $1 <= i <= t$. Write an algorithm (using *C* pseudocode) to store two sparse matrices (taken as input from the user) in the form (as specified above using X), perform addition of these matrices, store the result in the form (as specified above using X) and display the result (in the form which is usually understood by an user unfamiliar with the representation specified using X).

b) Specify the output of the following recursive function for the call tower(4, 'A', 'B', 'C');

```
int tower(int n, char source, char inter, char dest)
{
  if(n>0)
                         C
   {           A         B      B
     tower(n-1, source, dest, inter);                 A      C
     printf("\n Move disk %d from %c to %c", n, source, dest);
     tower(n-1, inter, source, dest);
   }                B     A     C
  return 0;
}
```

(5+5)

***